Asheley Mudzingwa

CIS 350

Transcendential Search Algorithm

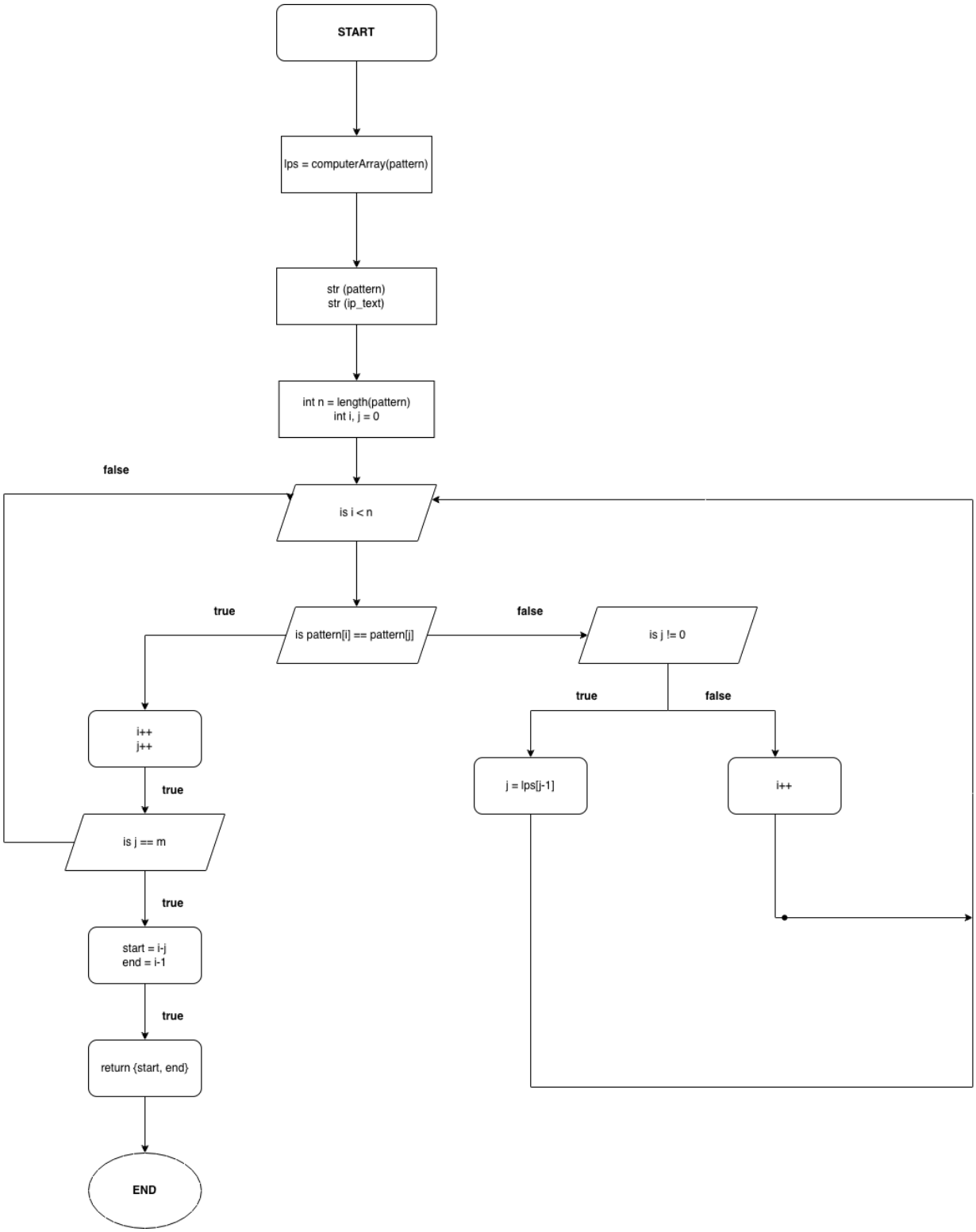
Github: https://github.com/amudzingwa/pi-search-algorithm

1. Pseudocode:

```
KMP-SEARCH(text, pattern, LPS)
n ← length(text)
m ← length(pattern)
i ← 0        // index for text
j ← 0        // index for pattern

WHILE i < n
   IF text[i] = pattern[j]
      i ← i + 1
      j ← j + 1

      IF j = m
         start ← i - j
         end ← i - 1
```

```
            RETURN (start, end)


        ELSE

            IF j != 0

                j ← LPS[j - 1]

            ELSE

                i ← i + 1


    END WHILE

    RETURN "Pattern Not Found"
```

2. Flowchart:


START
lps = computerArray(pattern)
str (pattern)
str (ip_text)
int n = length(pattern)
int i, j = 0
false
is i < n
true
is pattern[i] == pattern[j]
false
is j != 0
true
false
i++
j++
j = lps[j-1]
i++
true
is j == m
true
start = i-j
end = i-1
true
return {start, end}
END

3. Test:

Birthday: 11092001

```
TERMINAL                                                    C/C++ Compile Run
mudzingwa@Asheleys-MacBook-Pro-2 output % cd "/Users/mudzingwa/Desktop/Spring 2026/pi_transendant/output"
mudzingwa@Asheleys-MacBook-Pro-2 output % ./"pi_transcendental"
Enter number sequence to search for
In numerical format: twenty january
Pattern not found!
20375627181 nanoseconds time taken to run algorithm
mudzingwa@Asheleys-MacBook-Pro-2 output % cd "/Users/mudzingwa/Desktop/Spring 2026/pi_transendant/output"
mudzingwa@Asheleys-MacBook-Pro-2 output % ./"pi_transcendental"
Enter number sequence to search for
In numerical format: 11092001
Pattern found at position: 188772714 - 188772721
4514566919 nanoseconds time taken to run algorithm
mudzingwa@Asheleys-MacBook-Pro-2 output %
Ln 87, Col 37   Spaces: 4   UTF-8   LF   { } C++   Go Live   Mac
```

4. Complexity Analysis:

josephusProb_naive.cpp | pi_transcendental.cpp | frequency_counter.cpp | divandconq.cpp

Users > mudzingwa > Desktop > Spring 2026 > pi_transendant > pi_transcendental.cpp > loadDigits(const string &)

```
vector<int> computeLPSArray(string &pattern) {
    while (i < n) {
        } else {
    }

    return lps;
}

// KMP Algorithm
vector<int> search(string &pat, string &txt) {
    int n = txt.length();
    int m = pat.length();

    // vector<int> lps(m);
    vector<int> res;
    vector<int> lps = computeLPSArray(pat);

    int i = 0;
    int j = 0;

    while (i < n) {
        if (txt[i] == pat[j]) {
            i++;
            j++;

            if (j == m) {
                int start = i - j;
                int end = i - 1;

                res.push_back(start);
                res.push_back(end);

                return {start, end};
            }
        }
        else {

            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }
    return res;
}
```

→ O(1)

→ O(1)

→ O(n)

→ O(1)

→ O(1)

Debug | Compile | Compile & Run | Ln 33, Col 2 | Spaces: 4 | UTF-8 | LF | C++ | Go Live | Mac

josephusProb_naive.cpp | pi_transcendental.cpp | frequency_counter.cpp | divandconq.cpp

Users > mudzingwa > Desktop > Spring 2026 > pi_transendant > pi_transcendental.cpp > loadDigits(const string &)

```
// Pi Digits file
string loadDigits(const string& filename) {
    ifstream file(filename);
    string digits, line;

    if (!file.is_open()) {
        cerr << "Failed to open file: " << filename << endl;
        return "";
    }

    while (getline(file, line)) {
        digits += line;
    }

    file.close();
    return digits;
}


// KMP Algorithm

vector<int> computeLPSArray(string &pattern) {
    int n = pattern.size();
    vector<int> lps(n, 0);

    int len = 0;
    int i = 1;

    while (i < n) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    return lps;
}
```

→ O(n)

→ O(1)

→ O(n)

→ O(1)

Debug | Compile | Compile & Run | Ln 33, Col 2 | Spaces: 4 | UTF-8 | LF | C++ | Go Live | Mac

Time complexity: 0(n)

Space complexity: 0(n)